🏠 Foto AI Agent - Self-Hosting Guide

Complete guide for deploying Foto AI Agent on your Ubuntu server from GitHub.

📋 Table of Contents

1. Prerequisites
2. Docker Installation
3. Quick Start
4. Configuration
5. Deployment
6. Cloudflare Tunnel Setup
7. Backup & Restore
8. Monitoring
9. Troubleshooting

🔧 Prerequisites

Server Requirements

- **OS**: Ubuntu 20.04 LTS or newer (Server or Desktop)
- **CPU**: 2+ cores
- **RAM**: 4GB minimum (8GB recommended)
- **Storage**: 20GB+ free space
- **Network**: Stable internet connection with public or home network access

Required Software

- **Git** - For cloning the repository
- **Docker Engine** 20.10+ - Container runtime
- **Docker Compose Plugin** 2.0+ - Multi-container orchestration

Required API Keys

These are already configured in the repository, but you'll need to verify them:

1. **Abacus.AI API Key** - For LLM functionality
   - Get yours at: https://abacus.ai

2. **SendGrid API Key** - For email notifications
   - Get yours at: https://sendgrid.com
   - Or use SMTP credentials from your email provider

3. **OpenWeatherMap API Key** - For weather-based recommendations
   - Get yours at: https://openweathermap.org/api

🐳 Docker Installation

For Ubuntu Server (Recommended)

Step 1: Update System

```
sudo apt update
sudo apt upgrade -y
```

Step 2: Remove Old Docker Versions

```
sudo apt remove docker docker-engine docker.io containerd runc
```

Step 3: Install Dependencies

```
sudo apt install -y \
    apt-transport-https \
    ca-certificates \
    curl \
    gnupg \
    lsb-release
```

Step 4: Add Docker's Official GPG Key

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
    sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
```

Step 5: Set Up Docker Repository

```
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-
archive-keyring.gpg] https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/
null
```

Step 6: Install Docker Engine

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin
```

Step 7: Enable Docker Service

```
sudo systemctl enable docker
sudo systemctl start docker
```

Step 8: Add Your User to Docker Group (Optional but recommended)

```
sudo usermod -aG docker $USER

# Log out and log back in, or run:
newgrp docker
```

Step 9: Verify Installation

```
docker --version
# Should output: Docker version 24.x.x or newer

docker compose version
# Should output: Docker Compose version v2.x.x or newer

# Test Docker
docker run hello-world
```

Quick Docker Installation (Alternative)

```
# One-line installation script
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

# Add current user to docker group
sudo usermod -aG docker $USER
newgrp docker
```

🚀 Quick Start

1. Clone the Repository

```
# Clone from GitHub
git clone https://github.com/sesal78/Foto-AI-Agent-DeepAgent.git

# Enter the project directory
cd Foto-AI-Agent-DeepAgent

# Check what's inside
ls -la
```

You should see:
- `Dockerfile` - Container build instructions
- `docker-compose.yml` - Service orchestration
- `.env.example` - Environment template
- `app/` - Next.js application
- `prisma/` - Database schema
- `scripts/` - Deployment and backup scripts

2. Configure Environment Variables

```
# Copy the environment template
cp .env.example .env

# Edit with your preferred text editor
nano .env
# Or: vim .env
# Or: code .env  (if using VS Code)
```

Required Configuration:

```
# Database Configuration
POSTGRES_USER=foto
POSTGRES_PASSWORD=your_strong_password_here  # ⚠️ Change this!
POSTGRES_DB=foto
DATABASE_URL=postgresql://foto:your_strong_password_here@postgres:5432/foto

# NextAuth Configuration
NEXTAUTH_URL=http://localhost:3000  # Change to your domain if using Cloudflare Tunnel
NEXTAUTH_SECRET=your_nextauth_secret_here  # Generate with: openssl rand -base64 32

# Abacus.AI LLM API
ABACUSAI_API_KEY=your_abacusai_api_key

# SendGrid Email Service
SENDGRID_API_KEY=your_sendgrid_api_key
FROM_EMAIL=noreply@yourdomain.com

# OpenWeatherMap API
OPENWEATHERMAP_API_KEY=your_openweather_api_key

# User Configuration
USER_EMAIL=your-email@example.com  # Your email for notifications
USER_NAME=Your Name
USER_LOCATION=Your City, State  # For weather and location-based challenges
```

Generate Secure Secrets:

```
# Generate NEXTAUTH_SECRET
openssl rand -base64 32

# Generate strong database password
openssl rand -base64 24
```

3. Deploy with Docker Compose

```
# Make scripts executable
chmod +x deploy.sh backup.sh restore.sh monitor.sh

# Deploy the application
./deploy.sh
```

The deploy script will automatically:
✅ Build Docker images
✅ Start PostgreSQL database

✅ Start the Next.js application
✅ Run database migrations
✅ Seed the database with your profile
✅ Start the cron scheduler for automated challenges

Expected Output:

```
🚀 Deploying Foto AI Agent...
Building Docker images...
Starting services...
Running database migrations...
Seeding database...
✅ Deployment complete!
```

4. Initialize Database

```
# Run database migrations
docker compose exec app npx prisma migrate deploy

# Seed with your photography profile
docker compose exec app npx prisma db seed
```

5. Verify Installation

```
# Check service status
docker compose ps

# Check health endpoint
curl http://localhost:3000/api/health

# Expected response:
# {
#   "status": "healthy",
#   "database": "connected",
#   "timestamp": "2024-11-20T..."
# }
```

6. Access Your Application

Once deployed, access your Foto AI Agent at:
- **Home**: http://localhost:3000
- **Dashboard**: http://localhost:3000/dashboard
- **Chat Interface**: http://localhost:3000/chat
- **About**: http://localhost:3000/about

Default Login Credentials:
- Email: As configured in `.env` (USER_EMAIL)
- Password: Set during first signup

⚙️ Advanced Configuration

Environment Variables Reference

Located in `.env` (project root):

```
# Database Configuration
POSTGRES_USER=foto
POSTGRES_PASSWORD=your_strong_password
POSTGRES_DB=foto
DATABASE_URL=postgresql://foto:your_password@postgres:5432/foto

# NextAuth Configuration
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET=generated_32_char_secret

# API Keys
ABACUSAI_API_KEY=your_abacusai_key
SENDGRID_API_KEY=your_sendgrid_key
OPENWEATHERMAP_API_KEY=your_openweather_key

# Email Configuration
FROM_EMAIL=noreply@yourdomain.com
USER_EMAIL=your-email@example.com

# User Profile (for personalized challenges)
USER_NAME=Your Name
USER_LOCATION=Your City, State
USER_TIMEZONE=America/Los_Angeles  # Your timezone
```

Photography Profile Configuration

Edit `scripts/seed.ts` to customize your photography profile:

```
// Your camera gear
const userGear = [
  "Fujifilm X-T5",
  "Ricoh GR IIIx",
  "Nikon FE2",
  "Pentax ME"
];

// Your preferred photography times
const schedule = {
  weekends: "Saturday mornings",
  weekdays: "Lunch breaks, after work",
  workFromHome: "Around the house/neighborhood"
};
```

Automated Challenge Schedule

Edit `crontab` to customize when you receive challenges:

```
# Saturday morning challenges at 7 AM Pacific
0 15 * * 6 wget -qO- http://app:3000/api/scheduled/daily-challenge

# Weekday lunch challenges at 11 AM Pacific
0 19 * * 1-5 wget -qO- http://app:3000/api/scheduled/daily-challenge

# Weather monitoring every 3 hours (for opportunity alerts)
0 */3 * * * wget -qO- http://app:3000/api/scheduled/weather-monitor

# Weekly progress report on Sunday at 8 PM Pacific
0 4 * * 1 wget -qO- http://app:3000/api/scheduled/weekly-report

# Daily streak manager at 11 PM Pacific (tracks your progress)
0 7 * * * wget -qO- http://app:3000/api/scheduled/streak-manager
```

Timezone Conversion:
- Times in crontab are in UTC
- Add 8 hours to PST (7 hours to PDT)
- Example: 7 AM PST = 15:00 UTC

Customize Your Schedule:

```
# Edit the crontab file
nano crontab

# Apply changes (requires redeployment)
docker compose restart scheduler
```

🌐 Cloudflare Tunnel Setup

Why Cloudflare Tunnel?

- **No Port Forwarding** - No need to expose your home network
- **Free SSL/HTTPS** - Automatic SSL certificates
- **DDoS Protection** - Cloudflare's security
- **Hide Your IP** - Keeps your home IP address private

Setup Steps

1. **Run the setup script:**

```
./cloudflare-tunnel-setup.sh
```

1. **Follow the prompts:**
   - Authenticate with your Cloudflare account
   - Enter your domain name (e.g., `myfoto.topues.com`)

2. **Verify tunnel:**

```
cloudflared tunnel info foto-ai-agent
```

1. **Access via domain:**
   - Your app is now accessible at `https://myfoto.topues.com`

Manual Setup (Alternative)

If you prefer manual setup:

```
# Install cloudflared
curl -L https://github.com/cloudflare/cloudflared/releases/latest/download/
cloudflared-linux-amd64 -o cloudflared
sudo mv cloudflared /usr/local/bin/
sudo chmod +x /usr/local/bin/cloudflared

# Authenticate
cloudflared tunnel login

# Create tunnel
cloudflared tunnel create foto-ai-agent

# Route DNS
cloudflared tunnel route dns foto-ai-agent myfoto.topues.com

# Run tunnel
cloudflared tunnel run foto-ai-agent
```

💾 Backup & Restore

Automatic Backups

Set up daily backups with cron:

```
# Edit crontab
crontab -e

# Add daily backup at 2 AM
0 2 * * * /path/to/Foto-AI-Agent-DeepAgent/backup.sh
```

Manual Backup

```
./backup.sh
```

This backs up:
- PostgreSQL database
- Uploaded photos
- Environment configuration

Backups are stored in `./backups/` with timestamp.

Restore from Backup

1. **List available backups:**

```
ls backups/ | grep foto_db
```

1. **Restore specific backup:**

```
./restore.sh 20241119_140000
```

⚠️ **Warning**: This will replace all current data!

Backup Retention

Default retention: 7 days. Customize:

```
RETENTION_DAYS=30 ./backup.sh
```

📊 Monitoring

View Logs

```
# View all service logs in real-time
docker compose logs -f

# View specific service logs
docker compose logs -f app       # Application logs
docker compose logs -f postgres  # Database logs
docker compose logs -f scheduler # Cron job logs

# View last 100 lines
docker compose logs --tail=100 app

# Search logs for errors
docker compose logs app | grep -i error
```

Check Service Status

```
# Check all running services
docker compose ps

# Check specific service health
docker compose ps app
docker compose ps postgres

# Detailed service info
docker inspect foto-ai-agent_app_1
```

Health Check Endpoints

```
# Application health
curl http://localhost:3000/api/health

# Expected response:
# {
#   "status": "healthy",
#   "timestamp": "2024-11-20T...",
#   "database": "connected",
#   "service": "foto-ai-agent"
# }

# Database connection test
docker compose exec postgres pg_isready -U foto
```

Monitor System Resources

```
# Real-time resource usage
docker stats

# Disk usage by Docker
docker system df

# Detailed volume usage
docker system df -v

# Container resource limits
docker compose top
```

Monitor Script

Run the included monitoring script:

```
./monitor.sh
```

This will show:
- ✅ Service status
- 📊 Resource usage (CPU, memory)
- 💾 Disk space
- 🗄️ Database status
- 📝 Recent logs

🔧 Maintenance

Update Application

```
# Navigate to project directory
cd ~/Foto-AI-Agent-DeepAgent

# Pull latest changes from GitHub
git pull origin master

# Rebuild and redeploy
./deploy.sh

# Or manually:
docker compose down
docker compose build --no-cache
docker compose up -d
docker compose exec app npx prisma migrate deploy
```

Restart Services

```
# Restart all services
docker compose restart

# Restart specific service
docker compose restart app
docker compose restart postgres
docker compose restart scheduler

# Force recreate containers
docker compose up -d --force-recreate
```

Clean Up Docker Resources

```
# Stop and remove containers (keeps volumes/data)
docker compose down

# Remove containers AND volumes (⚠️ DELETES ALL DATA!)
docker compose down -v

# Clean up unused Docker resources
docker system prune -a

# Remove dangling images
docker image prune

# Remove unused volumes
docker volume prune
```

Database Management

```
# Access PostgreSQL CLI
docker compose exec postgres psql -U foto -d foto

# Common SQL queries:
# View all users
SELECT * FROM "User";

# View challenges
SELECT * FROM "DailyChallenge" ORDER BY "createdAt" DESC LIMIT 10;

# View user progress
SELECT * FROM "UserProgress";

# View submissions
SELECT * FROM "Submission" ORDER BY "submittedAt" DESC;

# Exit PostgreSQL
\q
```

Database Backup (Manual)

```
# Create timestamped backup
docker compose exec postgres pg_dump -U foto foto > backup_$(date +%Y%m%d_%H%M%S).sql

# Restore from backup
docker compose exec -T postgres psql -U foto foto < backup_20241120_120000.sql
```

🐛 Troubleshooting

Application Won't Start

1. **Check logs:**
   `bash`
   `docker-compose logs app`
2. **Verify database:**
   `bash`
   `docker-compose exec postgres pg_isready -U foto`
3. **Check environment:**
   `bash`
   `cat nextjs_space/.env`

Database Connection Issues

```
# Restart database
docker-compose restart postgres

# Check database logs
docker-compose logs postgres

# Verify connection string
echo $DATABASE_URL
```

Cron Jobs Not Running

1. **Check scheduler logs:**
   `bash`
   `docker compose logs scheduler`

2. **Verify crontab configuration:**
   `bash`
   `cat crontab`

3. **Test scheduled endpoints manually:**
   ```bash
   # Test daily challenge
   curl http://localhost:3000/api/scheduled/daily-challenge

# Test weather monitor
curl http://localhost:3000/api/scheduled/weather-monitor

# Test weekly report
curl http://localhost:3000/api/scheduled/weekly-report
```

1. **Restart scheduler:**
   `bash`
   `docker compose restart scheduler`

Email Notifications Not Sending

1. **Verify SendGrid API key:**
   `bash`
   `grep SENDGRID_API_KEY .env`

2. **Check email-related logs:**
   `bash`
   `docker compose logs app | grep -i "email\|sendgrid\|mail"`

3. **Test email functionality:**
   ```bash
   # Trigger a challenge (which sends email)
   curl -X POST http://localhost:3000/api/scheduled/daily-challenge
   ```

```
# Check logs immediately after
docker compose logs –tail=50 app | grep -i email
```

1. **Verify SendGrid account:**
   - Check your SendGrid dashboard for bounces/errors
   - Verify sender email is authenticated
   - Check API key permissions

Port Already in Use

```
# Find process using port 3000
lsof -i :3000

# Kill process
kill -9 <PID>

# Or use different port in docker-compose.yml
# Change "3000:3000" to "8080:3000"
```

Out of Disk Space

```
# Clean up Docker
docker system prune -a --volumes

# Remove old backups
find backups/ -mtime +30 -delete

# Check disk usage
df -h
```

📱 Mobile Access

Local Network Access

1. Find your server's local IP:
   `bash`
   `ip addr show | grep "inet " | grep -v 127.0.0.1`
2. Access from mobile:
   - `http://192.168.x.x:3000`

Internet Access (with Cloudflare Tunnel)

- `https://myfoto.topues.com`

🔐 Security Best Practices

1. **Change default passwords:**
   - Update `POSTGRES_PASSWORD` in `.env`
   - Generate strong `NEXTAUTH_SECRET`

2. **Keep Docker updated:**
   ```bash
   sudo apt update && sudo apt upgrade docker-ce docker-compose-plugin
   ```

3. **Enable firewall:**
   ```bash
   sudo ufw allow 22/tcp    # SSH
   sudo ufw allow 3000/tcp  # App (only if not using Cloudflare Tunnel)
   sudo ufw enable
   ```

4. **Regular backups:**
   - Set up automated daily backups
   - Test restore process monthly

5. **Monitor logs:**
   - Check logs regularly for errors
   - Set up alerts for critical issues

📞 Support & Resources

Documentation

- **This Computer Usede**: Complete self-hosting instructions
- **README.md**: Project overview and quick start
- **AI_IMPLEMENTATION_PROMPT.md**: Technical implementation details
- **FOTO_BUILD_FROM_SCRATCH_PROMPT.md**: From-scratch build instructions

GitHub Repository

- **Repository**: https://github.com/sesal78/Foto-AI-Agent-DeepAgent
- **Report Issues**: Create an issue on GitHub for bugs or feature requests
- **Pull Requests**: Contributions welcome!

Getting Help

1. **Check logs first:**
   ```bash
   docker compose logs app | tail -100
   ```

2. **Verify configuration:**
   ```bash
   cat .env | grep -v "PASSWORD\|SECRET\|KEY"
   ```

3. **Test health endpoint:**
   ```bash
   curl http://localhost:3000/api/health
   ```

4. **Review documentation:**
   - Read this Computer Usede thoroughly
   - Check GitHub issues for similar problems
   - Review error messages in logs

Common Commands Quick Reference

```
# Service Management
docker compose ps                     # Check status
docker compose logs -f                # View real-time logs
docker compose restart                # Restart all services
docker compose restart app            # Restart specific service

# Data Management
./backup.sh                           # Create backup
./restore.sh <backup_dir>             # Restore from backup
docker compose exec postgres psql     # Access database

# Monitoring
./monitor.sh                          # Run monitoring script
docker stats                          # Resource usage
docker compose top                    # Process list

# Maintenance
git pull origin master                # Update code
./deploy.sh                           # Redeploy application
docker system prune -a                # Clean up Docker
```

System Information

To report issues, include this information:

```
# Docker version
docker --version
docker compose version

# System info
uname -a
cat /etc/os-release

# Service status
docker compose ps

# Recent logs
docker compose logs --tail=50 app
```

🎉 Success!

Your Foto AI Agent is now self-hosted and running on your Ubuntu server!

Quick Access Links

- **Home Page**: http://localhost:3000
- **Dashboard**: http://localhost:3000/dashboard
- **Chat Interface**: http://localhost:3000/chat
- **About**: http://localhost:3000/about

What's Next?

1. ✅ **Set up Cloudflare Tunnel** for external access

2. ✅ **Configure automated backups** (cron job)
3. ✅ **Customize your challenge schedule** (edit `crontab`)
4. ✅ **Submit your first photo** and track progress!

Your AI Photography Coach Awaits! 📸

The system is now actively monitoring:
- 🌤️ Weather conditions for optimal shooting times
- 📅 Your schedule for challenge timing
- 📍 Your location for site recommendations
- 📊 Your progress and skill development

Start your photography journey today and watch your skills grow with personalized, AI-powered coaching!

Made with ❤️ for photographers who want to level up their craft